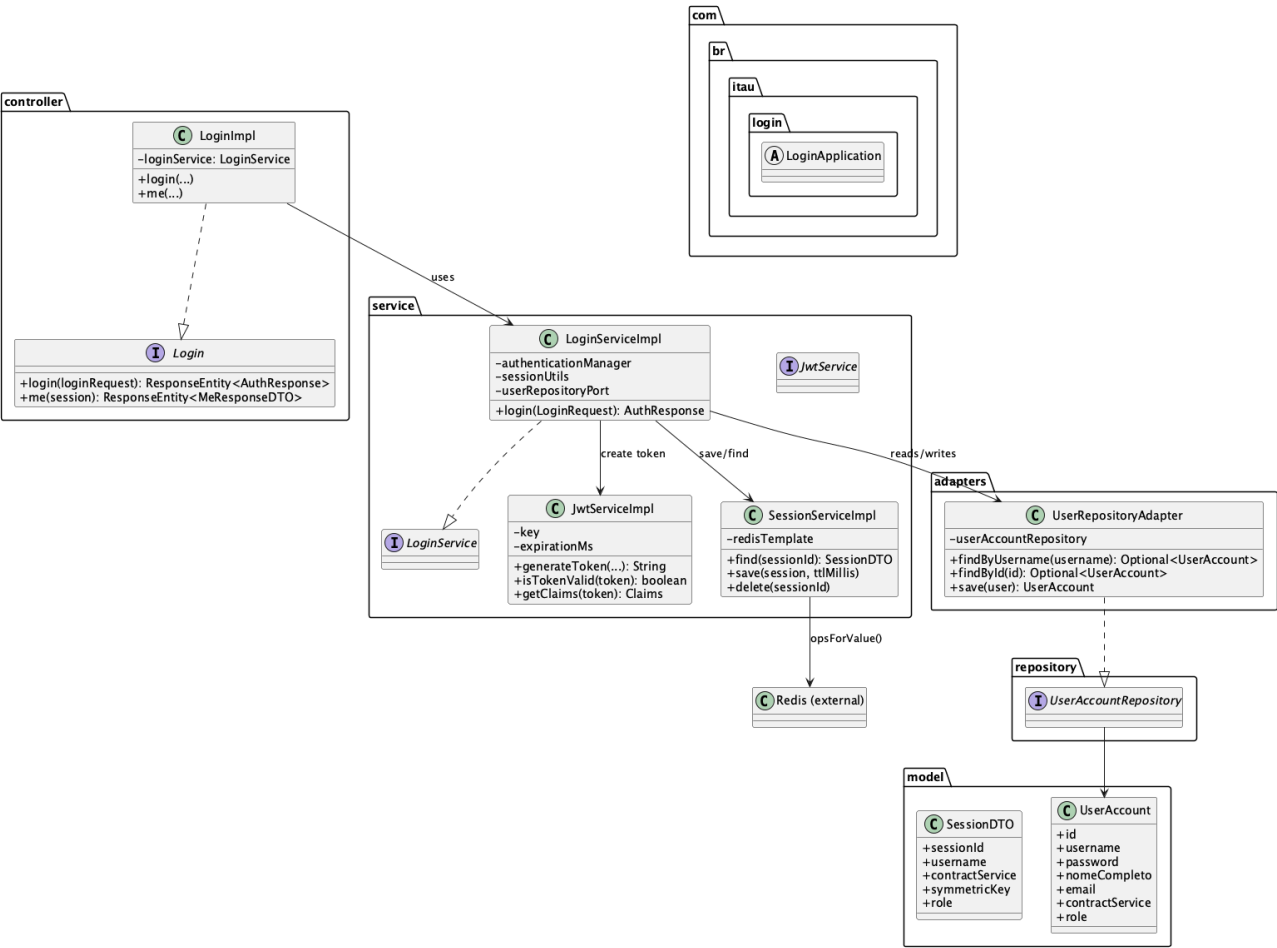
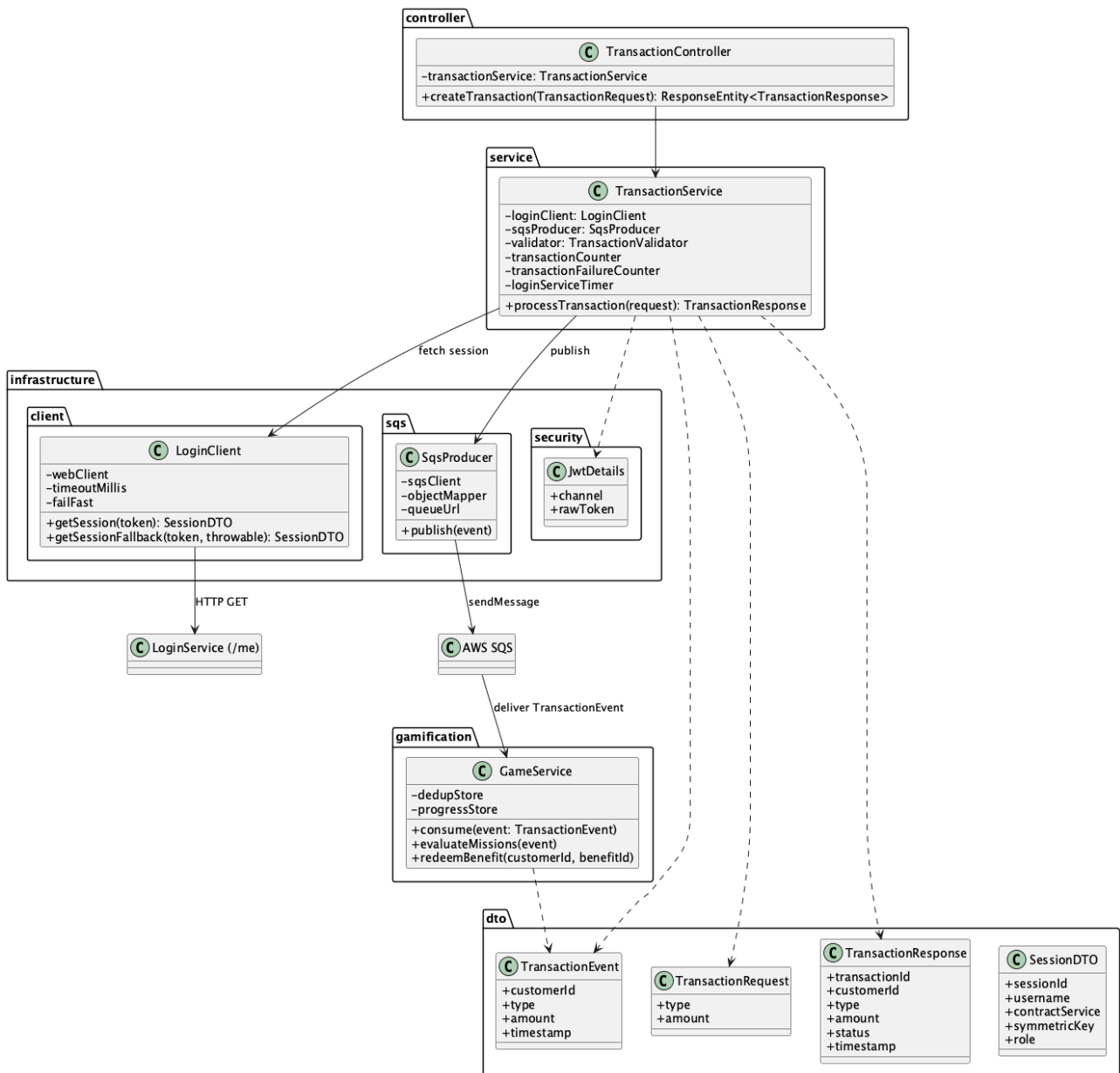


# Documentação Técnica — LoginService & TransactionService

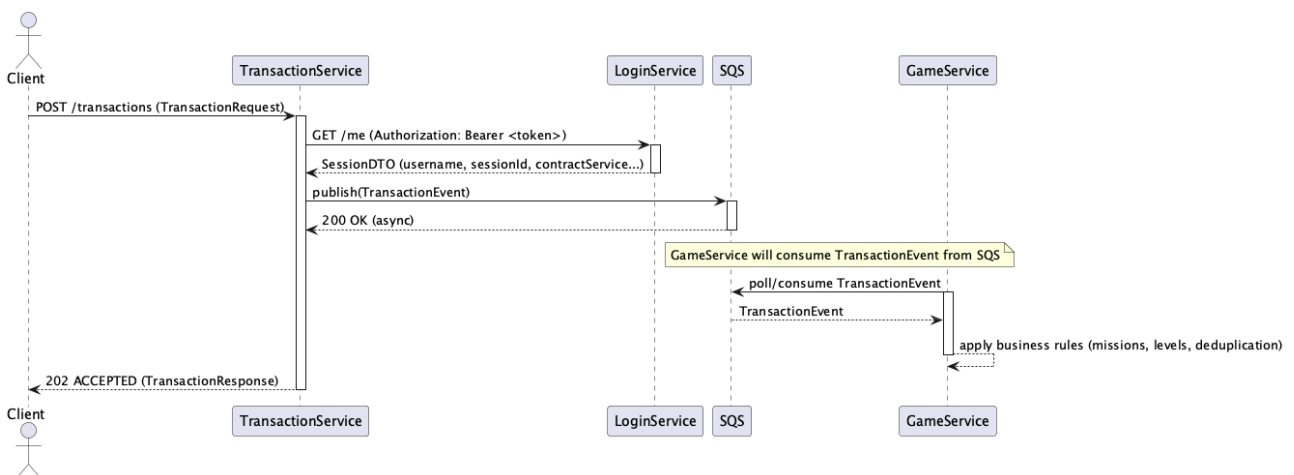
## LoginService - Diagrama de Classes



## TransactionService - Diagrama de Classes



## Diagrama de Sequência



# Documentação (PT-BR)

## Documentação dos códigos — PT-BR

Este documento descreve as partes principais do LoginService e do TransactionService, com explicações por classe e diagramas de classes (arquivos PlantUML incluídos).

### Instruções rápidas

- Diagramas: login\_service.puml e transaction\_service.puml (na mesma pasta). - Para gerar PDF (recomendado em macOS): instale pandoc e plantuml (ou use o script generate\_pdf.sh).

---

### Sumário (LoginService)

1. com.br.itau.login.LoginApplication — ponto de entrada Spring Boot. 2. controller.Login (interface) e controller.LoginImpl — endpoints /auth/login e /auth/me. 3. service.LoginService / service.LoginServiceImpl — lógica de autenticação: autentica via AuthenticationManager, recupera UserAccount, gera sessão (sessionId, symmetricKey) e gera token JWT via JwtService. 4. service.JwtService / service.JwtServiceImpl — encapsula geração e validação de JWTs (usa jjwt e uma chave HMAC configurada por jwt.secret). 5. service.SessionServiceImpl — consulta/grava sessões em Redis (usa RedisTemplate). 6. adapters.UserRepositoryAdapter + repository.UserAccountRepository + model.entity.UserAccount — persistência JPA do usuário. 7. model.SessionDTO — DTO usado para representar sessão armazenada no Redis e no token.

### Observações por arquivo/classe (resumo)

- LoginImpl (controller): recebe LoginRequest e delega para LoginService.login(...). Em /me expõe dados da sessão a partir do @AuthenticationPrincipal.

- LoginServiceImpl: realiza a autenticação via AuthenticationManager.authenticate(...), busca dados do usuário via UserRepositoryDomain, gera sessionId e symmetricKey via SessionUtils, salva a sessão em Redis e cria o token JWT chamando SessionUtils.createToken(...).

- JwtServiceImpl: gera token com claims sessionId, role e contractService. Valida assinaturas com a chave configurada (jwt.secret) e oferece método para extrair Claims.

- SessionServiceImpl: lê valor do Redis e converte para SessionDTO considerando que o Redis pode deserializar como SessionDTO ou como Map (tratamento robusto). Também salva com TTL usando ops.set(key, value, Duration).

- UserAccount (entity): campos principais: id, username, password, nomeCompleto, email, contractService, role (enum Role).

---

### Sumário (TransactionService)

1. com.transactionservice.TransactionServiceApplication — ponto de entrada Spring Boot. 2. controller.TransactionController — endpoint POST /transactions que recebe TransactionRequest e retorna TransactionResponse. 3. service.TransactionService — processamento principal: validação, verificação do canal (apenas MOBILE), consulta de sessão ao LoginService via LoginClient, publicação do evento em SQS via SqsProducer, métricas (Micrometer) e tratamento de falhas. 4. infrastructure.client.LoginClient — cliente WebClient para chamar o endpoint /me do LoginService, com CircuitBreaker e Retry (Resilience4j) e estratégia fail-fast / fail-open. 5. infrastructure.sqs.SqsProducer — serializa TransactionEvent para JSON e publica em SQS (AWS SDK v2 SqsClient). 6. dto.SessionDTO — record que espelha a sessão

retornada pelo LoginService. 7. infrastructure.security.JwtDetails — record que guarda informações extraídas do JWT para uso no Authentication object.

## Observações por arquivo/classe (resumo)

- TransactionService.processTransaction(...): verifica Authentication no SecurityContextHolder, obtém customerId e JwtDetails, valida o canal, chama fetchAndValidateSession(...) que usa LoginClient.getSession(...) (remota), valida contractService e publica o evento em SQS.
- LoginClient: configura WebClient (qualificado loginServiceWebClient), tem timeout configurável, e fallback com Resilience4j que, dependendo de feature-flags.login-service-fail-fast, bloqueia (lança LoginServiceUnavailableException) ou permite (retorna null) — chamando atenção ao comportamento de segurança (risco se fail-open).
- SqsProducer: converte TransactionEvent em JSON com ObjectMapper, invoca sqsClient.sendMessage(...) e registra logs e tratamento de exceções.

---

## Diagrama de Sequência

O diagrama abaixo ilustra o fluxo básico de uma transação entre sistemas:



Fluxo resumido: - Cliente -> TransactionService (POST /transactions) - TransactionService -> LoginService (/me) para validar sessão - TransactionService -> SQS (publica TransactionEvent) - GameService -> SQS (consome eventos para gamificação) — módulo ainda em desenvolvimento

> Observação: a parte de gamificação (GameService) está planejada, ainda a ser implementada. Ela consumirá TransactionEvent da fila SQS e aplicará regras de missões e níveis.

## Regras de Negócio (exemplo de alto nível)

- Missão por range: definir missões que são concluídas ao atingir um intervalo de valores (e.g., 5 transações PIX entre 10 e 100 BRL)
- Evitar duplicidade: garantir que um mesmo TransactionEvent.eventId não seja processado mais de uma vez
- Reset mensal: zerar contadores/estatísticas de missões no início de cada mês
- Níveis: atribuir pontos e níveis ao usuário com base em transações e missões concluídas

## GameService (proposto)

- Consome eventos do SQS (TransactionEvent).
- Avalia missões por tipo e range (e.g., contar transações PIX entre intervalos de valor).
- Controla duplicidade usando eventId para evitar processamento repetido.
- Mantém progresso por cliente (estado de missões, pontos, saldo de resgates).
- Calcula nível mensal com base em pontos acumulados e reseta contadores conforme a regra de reset mensal.
- Controla resgate de benefício: valida elegibilidade, decrementa saldo e registra transações de resgate.

> Observação: o GameService é um componente proposto para gamificação; arquitetura e detalhes de persistência (Redis, DynamoDB, RDS) devem ser definidos conforme requisitos de escalabilidade e latência.

## TransactionEvent (exemplo JSON)

```
{ "eventId": "uuid", "customerId": "...", "type": "PIX", "amount": 100, "timestamp": "...", "channel": "MOBILE" }
```

## Resiliência e estratégias

- Circuit Breaker: protege a aplicação contra falhas repetidas do LoginService, acionando fallback quando necessário. - Retry: re-tenta chamadas transitórias antes de acionar fallback. - Fail Fast vs Fail Open: comportamento configurável no LoginClient — em modo Fail Fast a indisponibilidade do LoginService bloqueia a transação; em Fail Open a transação segue sem validação (com risco).

---

## Como gerar o PDF (recomendado em macOS)

1. Instale dependências via Homebrew (se não tiver):

```
brew install pandoc brew install plantuml # opcional; o script pode baixar plantuml.jar se não houver
```

2. No diretório docs/ execute o script generate\_pdf.sh (ele tenta renderizar os diagramas e então chamar pandoc para converter os Markdown em PDF, incorporando as imagens).

Se preferir, use o script generate\_pdf\_reportlab.py que gera um PDF simples sem exigir LaTeX.

---

Fim do documento PT-BR.

# Documentation (EN)

## Code documentation — EN

This document describes the main parts of LoginService and TransactionService, with class explanations and PlantUML class diagrams included.

## Quick instructions

- Diagrams: login\_service.puml and transaction\_service.puml (in the same folder). - To generate a PDF (recommended on macOS): install pandoc and plantuml (or use the generate\_pdf.sh script).

---

## Summary (LoginService)

1. com.br.itaau.login.LoginApplication — Spring Boot entry point. 2. controller.Login (interface) and controller.LoginImpl — endpoints /auth/login and /auth/me. 3. service.LoginService / service.LoginServiceImpl — authentication flow: authenticate using AuthenticationManager, loads UserAccount, creates session (sessionId, symmetricKey) and generates JWT token via JwtService. 4. service.JwtService / service.JwtServiceImpl — handles JWT generation and validation (jjwt, HMAC key configured via jwt.secret). 5. service.SessionServiceImpl — reads/writes sessions in Redis using RedisTemplate. 6. adapters.UserRepositoryAdapter + repository.UserAccountRepository + model.entity.UserAccount — JPA persistence for users. 7. model.SessionDTO — DTO used to represent session stored in Redis and in tokens.

## Notes by file/class (summary)

- LoginImpl (controller): receives LoginRequest and delegates to LoginService.login(...). /me returns session data from @AuthenticationPrincipal.
- LoginServiceImpl: authenticates via AuthenticationManager.authenticate(...), retrieves user via UserRepositoryDomain, generates sessionId and symmetricKey via SessionUtils, saves session to Redis and creates JWT by calling SessionUtils.createToken(...).
- JwtServiceImpl: creates token with claims sessionId, role and contractService. Validates signature with configured jwt.secret and exposes method to extract Claims.
- SessionServiceImpl: reads values from Redis and attempts to convert them to SessionDTO either directly or from a Map when a JSON serializer is used. Saves with TTL.
- UserAccount (entity): main fields: id, username, password, nomeCompleto, email, contractService, role (enum Role).

---

## Summary (TransactionService)

1. com.transactionservice.TransactionServiceApplication — Spring Boot entry point. 2. controller.TransactionController — POST /transactions endpoint that receives TransactionRequest and returns TransactionResponse. 3. service.TransactionService — core processing: validation, channel check (only MOBILE), session fetch from LoginService via LoginClient, publish event to SQS via SqsProducer, metrics (Micrometer) and error handling. 4. infrastructure.client.LoginClient — WebClient-based client to call /me endpoint of LoginService, with Resilience4j CircuitBreaker+Retry and configurable fail-fast/fail-open behavior. 5. infrastructure.sqs.SqsProducer — converts TransactionEvent to JSON and publishes to AWS SQS (SqsClient). 6. dto.SessionDTO — record that mirrors the session returned by LoginService. 7. infrastructure.security.JwtDetails — record keeping JWT-derived info for the

Authentication object.

## Notes by file/class (summary)

- TransactionService.processTransaction(...): checks Authentication from SecurityContextHolder, gets customerId and JwtDetails, validates channel, calls fetchAndValidateSession(...) which uses LoginClient.getSession(...) (remote), validates contractService and publishes the event to SQS.
- LoginClient: configures loginServiceWebClient, timeout and fallback policy. Depending on feature-flags.login-service-fail-fast, either throws LoginServiceUnavailableException or returns null (fail-open).
- SqsProducer: serializes TransactionEvent and calls sqsClient.sendMessage(...) with logging and exception handling.

---

## Sequence Diagram

The following sequence diagram shows the typical flow for a transaction:



Summary flow: - Client -> TransactionService (POST /transactions) - TransactionService -> LoginService (/me) to validate session - TransactionService -> SQS (publish TransactionEvent) - GameService -> SQS (consume events for gamification) — module still under development

Note: Gamification (GameService) is planned and will consume TransactionEvent from SQS to update missions, points and levels.

## Business Rules (high-level examples)

- Mission by range: missions that complete when a user reaches a range of transaction values (e.g., 5 PIX txns between 10 and 100 BRL)
- Avoid duplication: ensure the same TransactionEvent.eventId is not processed twice
- Monthly reset: reset mission counters/statistics at the start of each month
- Levels: assign points and levels to users based on transactions and completed missions

## GameService (proposed)

- Consumes events from SQS (TransactionEvent).
- Evaluates missions by type and range (e.g., count PIX transactions within value ranges).
- Controls duplication using eventId to avoid re-processing the same event.
- Maintains per-customer progress (missions state, points, redemption balances).
- Calculates monthly level based on accumulated points and resets counters according to the monthly reset rule.
- Controls benefit redemption: validates eligibility, decrements balances and records redemption transactions.

Note: GameService is a proposed gamification component; persistence choices (Redis, DynamoDB, RDS) should be decided based on scalability and latency requirements.

## TransactionEvent (JSON example)

```
{ "eventId": "uuid", "customerId": "...", "type": "PIX", "amount": 100, "timestamp": "...", "channel": "MOBILE" }
```

## Resilience and strategies

- Circuit Breaker: protects the application from repeated failures of the LoginService, triggering fallbacks when needed. - Retry: retries transient calls before falling back. - Fail Fast vs Fail Open: configurable behavior in LoginClient — Fail Fast blocks transactions when LoginService is unavailable; Fail Open allows transactions to proceed without validation (risk).

---

## How to generate the PDF (recommended on macOS)

1. Install dependencies with Homebrew (if you don't have them):

```
brew install pandoc brew install plantuml # optional; the script can download plantuml.jar if missing
```

2. In the docs/ folder run the script generate\_pdf.sh (it renders diagrams and runs pandoc to create a PDF from Markdown including images).

Alternatively, use generate\_pdf\_reportlab.py to create a simple PDF without LaTeX.

---

End of EN document.